

Implementando una API rest

M8: IMPLEMENTACIÓN DE API BACKEND NODE EXPRESS

|AE2: IMPLEMENTAR UN SERVIDOR REST UTILIZANDO EL FRAMEWORK EXPRESS PARA LA DISPONIBILIZACIÓN DE RECURSOS ACORDE A LAS BUENAS PRÁCTICAS.

Introducción

En el desarrollo backend con JavaScript, el framework **Express** se ha consolidado como una de las herramientas más utilizadas para construir servidores web de manera rápida, flexible y minimalista. Express permite implementar APIs RESTful siguiendo buenas prácticas de diseño, facilitando la definición de rutas, el manejo de peticiones HTTP y la generación de respuestas con estructuras claras y coherentes.

Este manual está orientado a introducir al estudiante en la creación de un servidor REST utilizando Express, desde la inicialización del proyecto hasta la implementación de rutas que gestionan recursos a través de los verbos HTTP más comunes: GET, POST, PUT y DELETE. A lo largo del material también se abordará la recepción de parámetros, el uso de `express.json()` para procesar datos del cuerpo de la petición, y el uso correcto de los códigos de estado HTTP.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Iniciar un proyecto backend con Node.js y Express.
- Definir rutas RESTful que respondan a diferentes tipos de peticiones HTTP.
- Implementar lógica de negocio para manejar recursos (crear, leer, actualizar, eliminar).
- Procesar parámetros de ruta, query y cuerpo.
- Enviar respuestas en formato JSON con el código HTTP adecuado.
- Aplicar buenas prácticas en la estructuración de los endpoints REST.

¿Qué es Express y cómo iniciar un proyecto?

¿Qué es Express?

Express es un framework minimalista para Node.js que permite crear aplicaciones web y APIs de forma simple y rápida. Aunque es liviano, es extremadamente poderoso y flexible, lo que lo convierte en una de las herramientas más populares para el desarrollo backend en JavaScript.

Se utiliza principalmente para:

- Crear servidores web.
- Construir APIs RESTful.
- Manejar rutas y peticiones HTTP.
- Procesar datos de formularios o JSON.
- Integrar middleware para seguridad, validación, logging, etc.



¿Cómo iniciar un proyecto con Express?

Paso 1: Crear la carpeta del proyecto

```
mkdir mi-api-rest  
cd mi-api-rest
```

Paso 2: Inicializar un proyecto Node.js

```
npm init -y
```

Esto crea un archivo **package.json** con la configuración base.

Paso 3: Instalar Express

```
npm install express
```



Paso 4: Crear el archivo principal

Creamos un archivo llamado **index.js**:

```
const express = require('express');  
const app = express();  
  
// Middleware para interpretar JSON  
app.use(express.json());
```

```
// Ruta de prueba
app.get('/', (req, res) => {
  res.send('¡Servidor funcionando!');
});

// Escuchar en un puerto
app.listen(3000, () => {
  console.log('Servidor escuchando en http://localhost:3000');
});
```

▶ Paso 5: Ejecutar el servidor

Desde la terminal:

```
node index.js
```

Y al abrir <http://localhost:3000> en el navegador, deberías ver:

```
¡Servidor funcionando!
```

Creación de rutas RESTful con Express

Una **ruta** o **endpoint** en Express define cómo responde el servidor a una solicitud HTTP en una URL específica, usando un **verbo HTTP** (**GET**, **POST**, **PUT**, **DELETE**, etc.).

¿Qué es un endpoint RESTful?

Un endpoint RESTful sigue la estructura:

```
VERBO + /recurso
```

Por ejemplo:

```
GET /usuarios
POST /usuarios
GET /usuarios/1
PUT /usuarios/1
DELETE /usuarios/1
```

Ejemplo práctico: CRUD básico con usuarios

Primero, en `index.js`, agregamos una estructura base para manejar usuarios en memoria:

```
let usuarios = [
  { id: 1, nombre: 'Ana' },
  { id: 2, nombre: 'Luis' }
];
```

♦ 1. Obtener todos los usuarios (GET)

```
app.get('/usuarios', (req, res) => {
  res.json(usuarios);
});
```

♦ 2. Obtener un usuario por ID (GET /usuarios/:id)

```
app.get('/usuarios/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const usuario = usuarios.find(u => u.id === id);

  if (usuario) {
    res.json(usuario);
  } else {
    res.status(404).json({ error: 'Usuario no encontrado' });
  }
});
```

♦ 3. Crear un nuevo usuario (POST)

```
app.post('/usuarios', (req, res) => {
  const nuevo = req.body;
  nuevo.id = usuarios.length + 1;
  usuarios.push(nuevo);
  res.status(201).json(nuevo);
});
```

♦ 4. Actualizar un usuario (PUT)

```
app.put('/usuarios/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = usuarios.findIndex(u => u.id === id);

  if (index !== -1) {
    usuarios[index] = { ...req.body, id };
    res.json(usuarios[index]);
  } else {
    res.status(404).json({ error: 'Usuario no encontrado' });
  }
});
```

♦ 5. Eliminar un usuario (DELETE)

```
app.delete('/usuarios/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const originalLength = usuarios.length;
  usuarios = usuarios.filter(u => u.id !== id);

  if (usuarios.length < originalLength) {
    res.status(204).send();
  } else {
    res.status(404).json({ error: 'Usuario no encontrado' });
  }
});
```

Recepción de parámetros y cuerpo de las peticiones

En Express, podés acceder a distintos tipos de datos que llegan en una solicitud HTTP:

- `req.params`: parámetros de ruta (`/usuarios/:id`)
- `req.query`: parámetros de consulta (`?orden=desc`)
- `req.body`: datos enviados en el cuerpo (por ejemplo, en un `POST`)

Parámetros de ruta (`req.params`)

Usados cuando una parte de la URL representa un valor variable.

Ejemplo:

```
app.get('/productos/:id', (req, res) => {
  const id = req.params.id;
  res.send(`Buscando el producto con ID ${id}`);
});
```

📌 Si accedés a `/productos/45`, la respuesta será:
Buscando el producto con ID 45

② Parámetros de consulta (`req.query`)

Se usan para enviar filtros u opciones adicionales en la URL.

Ejemplo:

```
app.get('/productos', (req, res) => {
  const categoria = req.query.categoria;
  const orden = req.query.orden;
  res.send(`Filtrando por categoría: ${categoria}, orden: ${orden}`);
});
```

📌 Si accedés a `/productos?categoria=libros&orden=precio_asc`, responderá:
Filtrando por categoría: libros, orden: precio_asc

3 Cuerpo de la petición (`req.body`)

Para leer el cuerpo de la solicitud (por ejemplo, en `POST` o `PUT`), usás el middleware `express.json()` para que Express pueda interpretar datos en formato JSON:

```
app.use(express.json()); // Esto debe estar al inicio
```

Ejemplo:

```
app.post('/usuarios', (req, res) => {
  const nuevoUsuario = req.body;
  res.status(201).json({
    mensaje: 'Usuario recibido',
    datos: nuevoUsuario
  });
});
```

📌 Si el cliente envía:

```
{
  "nombre": "Luna",
  "email": "luna@example.com"
}
```

📎 El servidor responde:

```
{
  "mensaje": "Usuario recibido",
  "datos": {
    "nombre": "Luna",
    "email": "luna@example.com"
  }
}
```

Códigos de respuesta HTTP

Cada vez que el servidor responde a una solicitud, lo hace con un **código de estado**, que indica el resultado de la operación. Estos códigos siguen el estándar HTTP y son interpretados por navegadores, apps móviles, y otros clientes.

En Express se devuelven con:

```
res.status(<código>).json({ ... });
```

1 2 3 4 Principales categorías de códigos

Categoría	Rango	Significado general
1xx	100–199	Informativos
2xx	200–299	Éxito
3xx	300–399	Redirecciones
4xx	400–499	Errores del cliente
5xx	500–599	Errores del servidor

✓ Códigos más usados en APIs RESTful

Código	Significado	Cuándo usarlo
200	OK	Petición exitosa (GET , PUT)
201	Created	Recurso creado (POST)
204	No Content	Eliminación exitosa (DELETE)
400	Bad Request	Datos inválidos
401	Unauthorized	No autenticado
404	Not Found	Recurso no existe
500	Internal Server Error	Error en el servidor

Ejemplos prácticos en Express

201 – Created:

```
app.post('/productos', (req, res) => {
  const producto = req.body;
  // guardar producto...
  res.status(201).json({ mensaje: 'Producto creado', data: producto });
});
```

400 – Bad Request:

```
app.post('/usuarios', (req, res) => {
  if (!req.body.nombre) {
    return res.status(400).json({ error: 'El nombre es obligatorio' });
  }
  // continuar...
});
```

404 – Not Found:

```
app.get('/usuarios/:id', (req, res) => {
  const usuario = usuarios.find(u => u.id ===
    parseInt(req.params.id));
  if (!usuario) {
    return res.status(404).json({ error: 'Usuario no encontrado' });
  }
  res.json(usuario);
});
```

Buenas prácticas en la estructuración de endpoints REST

Diseñar endpoints RESTful no solo se trata de que funcionen, sino de que sean **predecibles, organizados y fáciles de mantener**. Aquí van las principales buenas prácticas que deberías seguir:

1. Usar sustantivos en plural para los recursos

 Correcto:

GET /usuarios

POST /productos

✗ Incorrecto:

GET /obtenerUsuario

POST /crearProducto

● 2. Aprovechar los verbos HTTP correctamente

No uses el verbo en la URL. El método HTTP ya indica la acción:

Acción	Método + Ruta
Listar	GET /clientes
Obtener uno	GET /clientes/:id
Crear	POST /clientes
Actualizar	PUT /clientes/:id
Eliminar	DELETE /clientes/:id

● 3. Usar códigos de estado adecuados

Siempre respondé con el **status** que indique qué pasó:

- 200 → OK
- 201 → Creado
- 400 → Datos incorrectos
- 404 → No encontrado
- 500 → Error del servidor

✓ Esto mejora la interoperabilidad y permite a los clientes reaccionar automáticamente a los resultados.

● 4. Respuestas coherentes y en formato JSON

Todas las respuestas deben tener **estructura consistente**. Por ejemplo:

```
{
  "mensaje": "Recurso creado exitosamente",
  "data": {
    "id": 3,
    "nombre": "Nuevo producto"
  }
}
```

● 5. Evitar rutas innecesarias o repetitivas

Mantené la API limpia:

- ❌ `/api/v1/crearUsuario`
- ✅ `/api/v1/usuarios` con método `POST`

● 6. Incluir versionado desde el inicio

Ejemplo:

```
/api/v1/usuarios
/api/v2/usuarios
```

Esto permite actualizar la API sin romper integraciones anteriores.

Cierre

En este manual aprendiste a construir una API RESTful desde cero utilizando el framework **Express**, uno de los pilares del desarrollo backend moderno con JavaScript. Exploramos paso a paso cómo iniciar un proyecto, crear rutas para los distintos verbos HTTP, procesar solicitudes con parámetros y cuerpo, y enviar respuestas con códigos de estado apropiados.

También incorporaste buenas prácticas para estructurar los endpoints REST, garantizando claridad, mantenibilidad y escalabilidad. Este enfoque no solo es útil para proyectos pequeños, sino también para aplicaciones profesionales que requieren consistencia y robustez.

Dominar Express te da una base sólida para avanzar hacia desarrollos más complejos, como autenticación, bases de datos, middlewares personalizados, validaciones, y más.

Referencias

- Express. (s.f.). Documentación oficial de Express.js. <https://expressjs.com/es/>
- Mozilla. (s.f.). HTTP status codes. MDN Web Docs. <https://developer.mozilla.org/es/docs/Web/HTTP/Status>
- Node.js. (s.f.). Documentación oficial. <https://nodejs.org/es/docs/>
- REST API Tutorial. (s.f.). REST API tutorial – Best practices. <https://restfulapi.net/>
- Postman. (2020, 6 de octubre). API design tips. Postman Blog. <https://blog.postman.com/>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Cap. 5). University of California, Irvine. https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

¡Muchas gracias!

Nos vemos en la próxima lección

